

Scenario:

Customer tries to transfer ₹50,000.

Transaction failed.

Error shown: "Transaction could not be completed."

Using Chain of thoughts:**Expected discovery summary from the below analysis:**

Session timeout

App cache issue

Form validation glitch

Stale token

Amount exceeds per-transaction or daily limit;

Beneficiary not verified/cooling period active

Insufficient balance, account frozen/dormant, KYC pending

OTP/2FA failure, expired session, invalid auth token

Timeout between app and core banking system; API gateway failure

Step 1: Restate the observed behavior

- Action: Customer initiates a transfer of ₹50,000
- Expected: Transaction should succeed (assuming valid balance, valid beneficiary, no restrictions)
- Actual: Transaction fails with a generic message: *"Transaction could not be completed."*
- Problem: The error message gives no actionable detail — this itself is a UX/QA defect worth flagging separately from the root cause.

Step 2: Break down possible failure layers

Think of this as an onion — work from the user's action inward to the backend.

Layer	Possible Cause
Client-side / UI	Session timeout, app cache issue, form validation glitch, stale token
Input/Business rules	Amount exceeds per-transaction or daily limit; beneficiary not verified/cooling period active
Account state	Insufficient balance, account frozen/dormant, KYC pending
Authentication/Authorization	OTP/2FA failure, expired session, invalid auth token
Network/Integration	Timeout between app and core banking system; API gateway failure
Backend/Core banking	Ledger posting failure, downstream service (e.g., NPCI/UPI rail, IMPS/NEFT gateway) down

Layer	Possible Cause
Third-party/External	Beneficiary bank unavailable, payment rail outage
Infra	Server error (5xx), database lock/deadlock, queue backlog

Step 3: Narrow down using evidence you'd gather

Ask (or check logs for):

1. Does it fail for all amounts, or only ₹50,000+? → points to a limit/threshold rule
2. Does it fail for this user only, or all users? → account-specific vs. systemic
3. Does it fail on all channels (app/web/branch) or just one? → client vs. backend
4. What does the backend log show at the timestamp? → look for HTTP status code, exception stack trace, or gateway timeout
5. Was money debited but not credited? → indicates a partial failure / reconciliation issue (high severity — treat as P1)
6. Is this reproducible? → intermittent vs. consistent failure changes root cause hypothesis

Step 4: Classify likely root cause buckets (QA triage)

- If reproducible for all users at ₹50,000+ → likely a business rule/limit validation bug (e.g., misconfigured threshold)
- If reproducible for one user only → account-level issue (KYC, freeze, limit override)
- If intermittent → infra/timeout/race condition, needs load/concurrency testing
- If debit happened without credit → critical reconciliation defect, escalate immediately, don't just log as "failed"

Step 5: What I'd do next as QA

- Reproduce with same amount, same user, same channel
- Check with a lower amount (e.g., ₹5,000) to isolate amount-based limits
- Pull backend/API logs for correlation ID or transaction reference
- Verify against test data: account status, KYC status, beneficiary status, daily limit config
- File defect with: steps to reproduce, expected vs actual, logs/screenshots, severity (especially flag if debit-without-credit)

Want me to turn this into a formal bug report template or a test case matrix (e.g., for regression testing transfer limits) you can reuse for eInvoice/ACES-style documentation?

Scenario :1: Search Functionality:

Scenario 1 — Search Functionality (E-commerce Product Search)

I – Instruction :

Generate comprehensive test cases for the product search feature, covering:

- Functional search (exact match, partial match, case-insensitivity, multi-word queries)
- Mandatory/required field validation (e.g., is search allowed with blank input, minimum character threshold before triggering auto-suggest, whitespace-only input, max character limit)
- Filters (single and combined) and sorting behavior
- Empty/invalid/special-character input handling
- Export of search results (if supported — formats like CSV/Excel/PDF, whether export respects applied filters/sort, export of partial vs. full result set, export button visibility/permission by role)
- Performance under load (large result sets, concurrent searches, response time SLA)

C – Context:

- The search bar sits in the e-commerce header, searching product name/category/brand/SKU. Minimum 2 characters required to trigger auto-suggest; blank search on submit shows all products (not an error). Search results support export to CSV/Excel (max 10,000 rows per export), respecting currently applied filters and sort order. Filters: price range, category, brand, rating, availability. Sort: relevance, price low-high/high-low, newest, top-rated. Results paginated at 20/page. Target response time: <2 seconds for datasets up to 1M products.

E-Example:

Example Sample test case:

Test Case ID	Title	Steps / Input	Expected Result	Priority
TC001	Valid keyword search	Search "wireless mouse"	Relevant results displayed, sorted by relevance by default	High
TC002	Empty search input	Click search with no input	Show validation message or all products, not an error/crash	Me

P – Persona

Act as a QA Engineer validating e-commerce cart logic, pricing accuracy, edge-case handling, any other impacts of this fix

O – Output Format

CSV: Test Case ID, Title, Category (Add/Remove/Update/Pricing/Coupon/Stock), Preconditions, Steps, Expected Result, Priority.

T-Tone

Precise and calculation-focused — no ambiguity in expected results (especially price/discount math). This is a business-critical module flagged by the client; treat coverage as zero-tolerance for production defects. Every test case must have a deterministic, verifiable pass/fail outcome — no vague or subjective expected results.

Scenario 2. File Upload

I – Instruction:

Generate comprehensive test cases for the file upload feature, covering:

- Functional upload (single file, multiple files, drag-and-drop, browse-and-select)
- Mandatory/required field validation (is upload mandatory before form submission, minimum 1 file required, field-level error if skipped)
- Supported file types and size limits (accepted formats, rejection of unsupported formats, max size per file, max total size/count per submission)
- Success/failure criteria (upload progress indicator, success confirmation, failure/retry behavior)
- Security/malicious file handling (executable files, double extensions, script-embedded files, virus/malware scan behavior)
- Performance under load (large file upload time, multiple concurrent uploads, slow network/interrupted upload)

C – Context:

The upload feature is part of a document-submission module (e.g., KYC/invoice attachment). Accepted formats: PDF, JPG, PNG, DOCX. Max size: 5MB per file, max 3 files per submission. Upload is mandatory before final form submission — form cannot be submitted with zero files attached. Files are scanned for malware server-side before being marked "accepted"; invalid type/size is rejected both client-side (before upload starts) and server-side (as a safeguard). Failed uploads must allow retry without losing already-uploaded files in the same session.

E – Example:

Example Sample test case:

Test Case ID	Title	Steps / Input	Expected Result	Priority
TC001	Valid PDF upload	Upload 2MB PDF file	File uploaded successfully, preview/thumbnail shown, success confirmation displayed	High
TC002	Submit form without uploading file	Leave upload field empty, click Submit	Form blocked with "File upload is mandatory" validation error	High

P – Persona

Act as a QA Engineer validating file-handling logic, upload security (malicious file scenarios), size/type restrictions, and any downstream impacts (e.g., form submission blocked, storage/DB record creation, malware-scan failure handling).

O – Output Format

CSV: Test Case ID, Title, Category (Functional/Mandatory-Field/Negative/Security/Performance), Preconditions, Steps, Expected Result, Priority.

T – Tone

Precise and security-conscious — no ambiguity in expected results (especially malicious file and mandatory-field scenarios). This is a business-critical module flagged by the client; treat coverage as zero-tolerance for production defects. Every test case must have a deterministic, verifiable pass/fail outcome — no vague or subjective expected results.

Scenario 3 — Shopping Cart

I – Instruction:

Generate comprehensive test cases for the shopping cart feature, covering:

- Functional cart actions (add to cart, remove item, update quantity via +/- or manual entry)
- Mandatory/required field validation (minimum quantity 1, cannot set quantity to 0 or negative, cannot add out-of-stock item)

- Price calculation validation (subtotal, tax, shipping, grand total recalculation on every add/remove/update)
- Coupon/discount scenarios (valid coupon, expired coupon, invalid code, minimum order value condition, single vs. stacked coupons, coupon removal)
- Out-of-stock handling (item goes out of stock while already in cart, partial stock available vs. requested quantity)
- Cart persistence (guest session vs. logged-in user, cart retained across login/logout, cart expiry)
- Performance/edge cases (large number of items in cart, concurrent updates from multiple tabs/devices)

C – Context:

The cart is part of an e-commerce platform. Cart persists across sessions for logged-in users (stored server-side) and is session-only for guests (stored client-side, merged into account cart on login). Price recalculates live on any quantity/item change, including tax (based on shipping address) and shipping fee (based on order value/weight thresholds). Coupons apply either a flat or percentage discount, with minimum order value conditions; only one coupon can be applied at a time unless explicitly stackable. Stock is validated at add-to-cart time and re-validated at checkout — if stock drops below requested quantity, the cart must flag it before payment.

E – Example:

Example Sample test case:

Test Case ID	Title	Steps / Input	Expected Result	Priority
TC001	Add in-stock item to cart	Add 1 qty of an in-stock item to cart	Item added, cart icon count updates, subtotal reflects item price	High
TC002	Apply expired coupon	Enter an expired coupon code at checkout	Show "Coupon expired" error, no discount applied, total remains unchanged	Medium

P – Persona

Act as a QA Engineer validating e-commerce cart logic, pricing accuracy, edge-case handling, and any other downstream impacts of this fix (e.g., checkout, inventory, order summary).

O – Output Format

CSV: Test Case ID, Title, Category (Add/Remove/Update/Pricing/Coupon/Stock), Preconditions, Steps, Expected Result, Priority.

T – Tone

Precise and calculation-focused — no ambiguity in expected results (especially price/discount math). This is a business-critical module flagged by the client; treat coverage as zero-tolerance for production defects. Every test case must have a deterministic, verifiable pass/fail outcome — no vague or subjective expected results.

Scenario : 4: Role-Based Access Control

I – Instruction:

Generate comprehensive test cases for role-based access control, covering:

- Functional permission validation per role (what each role can view/create/edit/delete)
- Role definitions and permission matrix (Admin, Manager, Viewer — or app-specific roles)
- UI-level enforcement (hidden/disabled buttons, menus, fields based on role)
- API-level enforcement (direct API calls bypassing UI, to confirm backend also blocks unauthorized actions)
- Unauthorized access scenarios (URL manipulation, direct object reference, privilege escalation attempts)
- Role assignment/change scenarios (mid-session role change, role removal, multiple roles per user if supported)
- Session/token behavior tied to role (session persists correct permissions, token doesn't leak elevated access)

C – Context:

The application has three roles — Admin (full CRUD + user management), Manager (create/edit, no delete, no user management), Viewer (read-only). Permissions are enforced both in the UI (elements hidden/disabled per role) and at the API level (every endpoint validates role/token server-side, independent of UI state). Role is tied to the user's session token; a role change requires re-login or token refresh to take effect. Unauthorized actions (UI bypass or direct API call) must return a 403 Forbidden and must not partially execute.

E – Example:

Example Sample test case:

Test Case ID	Title	Steps / Input	Expected Result	Priority
TC001	Admin deletes a record	Login as Admin, delete an existing record	Record deleted successfully, confirmation shown, record removed from list	High
TC002	Viewer attempts delete via direct API call	Login as Viewer, send DELETE request directly to API (bypassing UI)	API returns 403 Forbidden, record remains unchanged in DB	High

P – Persona

Act as a Security-focused QA Engineer validating permission boundaries, UI vs. API enforcement consistency, and any downstream impacts of unauthorized access attempts (e.g., logs/alerts triggered, no partial data change).

O – Output Format

CSV: Test Case ID, Role, Title, Category (UI-Enforcement/API-Enforcement/Unauthorized-Access/Session), Preconditions, Steps, Expected Result, Priority. Include a separate Permission Matrix table (Role × Action: Allowed/Denied) as a companion reference.

T – Tone

Precise and security-conscious — no ambiguity in expected results, especially around what "denied" looks like (error shown vs. silent failure vs. partial execution). This is a business-critical module flagged by the client; treat coverage as zero-tolerance for production defects, since unauthorized access is a security risk, not just a functional bug. Every test case must have a deterministic, verifiable pass/fail outcome.